

L SYSTEMS

Implementazione in raylib e c++

B. Rodolfo, H.Kirollos

<https://github.com/alonzo-bazaar/lsystems>

CONTENTS

L Systems

(scusa rod se sto facendo
il prototipo in italiano)

Per disegnare un albero tramite L-system serve avere

- Un alfabeto di **simboli** da interpretare come istruzioni per disegnare l'albero questi simboli possono essere
 - **non parametrici**: una “lettera e basta”
 - **parametrici** una “lettera” accompagnata da un vettore di parametri ($\in \mathbb{R}^n$ per un qualche $\mathbb{N}$)
- Regole di **riscrittura** per stringhe fatte di questi simboli queste regole possono essere
 - **deterministiche** o **stocastiche**
 - **context free** (cosiddetti *0L-systems*) o context dependant (quale ad esempio un *2L-system*)
- Un sistema di **turtle graphics** a cui passare una stringa di questi simboli/istruzioni dopo che è passata tot volte per le regole di riscrittura

Noi si è realizzato uno **stochastic parametric OL System**

- **OL System:** Le regole di riscrittura (o *produzioni*, in gergo informatica teorica) sono *context free*, sarebbe a dire, determiniamo come sostituire un simbolo guardando solo quel simbolo, ignorando eventuali simboli che questo possa avere a destra o a sinistra all'interno della stringa
- **Stochastic:** Durante una riscrittura, la produzione che andiamo ad applicare al simbolo può essere fissa o può venire campionata da una qualche distribuzione di produzioni.
- **Parametric:** I simboli su cui vanno ad agire queste produzioni sono coppie lettera - vettore di parametri

Turtle Graphics

Per parlare della parte di riscrittura serve parlare dei simboli che compongono le stringhe di istruzioni, e per parlare di quei simboli bisogna contestualizzare su cosa vanno ad agire questi simboli.

Turtle graphics tradizionalmente composto da

- una posizione in $\mathbb{R}^2$ e un angolo
- una “penna” che si può alzare o abbassare per iniziare/smettere di disegnare

Un tale sistema prevede un set relativamente ridotto di istruzioni, solitamente

- vai avanti/dietro una certa lunghezza
- gira a destra/sinistra di un certo angolo
- abbassa o alza la “penna” dal “foglio”

Il sistema presentato nel libro prevede però ulteriori istruzioni, quali

- spostarsi in 3d e girare pitch, yaw, e roll
- salva stato precedente/ripristina un stato precedente salvato
- assottiglia/ispessisci il tratto (avanza l'indice in una tabella di spessori) (tronco spesso, rametti sottili)
- passa a un colore più scuro/più chiaro (avanza l'indice in una tabella di colori) (tronco marrone scuro, rametti verde chiaro)
- non è presente il concetto di una penna alzata/abbassata, andare avanti con o senza il “tracciare una riga” sono due istruzioni diverse
- { e } rinchiudono un contesto, tipo un `with` di python
 - { qua inizia un poligono, fino alla fine del poligono segnati tutti i punti da cui passi
 - } prendi tutti i punti che ti sei segnato e fatti un poligono

Per implementare un sistema che potesse eseguire queste istruzioni la nostra tartaruga ha

- posizione $\in \mathbb{R}^3$, angolo rappresentato come matrice 3×3 HLU (colonne formano vettori Heading, Left, Right)
- una tabella di spessori e un indice in questa tabella
- tabella di colori, sempre con indice, realizzata però come tabella di texture coordinate
 - facilita la fase di texturing il lavoro con lo shader (non bisogna gestire vertex color)
 - comportamento “color table” simulato con texture coordinate crescenti e una texture gradiente
- uno stack di stati precedentemente salvati, salvare fa push ripristinare fa pop
 - lo stato include anche gli indici nelle tabelle di spessore e texcoord

- flag per vedere se siamo in un “blocco” { ... } e un buffer di vertici dove ci segniamo il poligono corrente
 - buffer dove vengono salvati i vertici del poligono percorso durante il blocco { ... }

Inoltre per motivi di efficienza invece di disegnare l'albero la turtle crea un modello di albero, che poi passiamo subito alla gpu e non dobbiamo rieseguire la tartaruga a ogni frame in cui si vede l'albero, per creare il modello utilizziamo

- un buffer (espandibile) di vertex coordinate
- un buffer (espandibile) di texture coordinate

(qui parti con pippone sul pattern builder) (quando incontri un'istruzione di disengare roba aggiungi la roba al buffer, poi alla fine fai get e hai tutta la roba che hai aggiunto nelle varie chiamate)

- un simbolo è una coppia carattere + vettore di numeri che rappresenta un'istruzione per il sistema di turtle graphics descritto sopra, l'alfabeto di simboli accettato dalla nostra turtle, ripreso dal libro, è
- `F|f + stride`: avanza di una lunghezza pari a `stride` lungo l'angolo di heading, lasciando (`F`) o non lasciando (`f`) dietro una traccia di questo avanzamento
- `&|.|.|.|. + angolo`: ruota di `+(sdd)` o `-(sss)` angolo lungo l'asse `H(...)`, `L(...)`, o `U(...)`
- `'!` (senza parametri): incrementa indice in...
- `[|]` (senza parametri): salva lo stato corrente `[` o ripristina lo stato precedentemente salvato `]`
- `{|}` (senza parametri): inizia `{` a disegnare un poligono, o finisci di disegnare un poligono `}`

roba

roba

Terrain

- Procedural terrain generation
- Based on Fractal Perlin Noise chunks
 - using non precalculated vectors
- Amplitude, frequency, lacunarity are some of the attribute that models the shape

- A function in the Terrain class take care of the chunks generation
 - Using a value of Render Distance
 - When the player move and a rendered chunk became outside the RD
 - The Management Function delete the chunk and deallocate memory
- Every chunk generated comes with a mesh and a bounding box
 - the bbox is used for Frustum Culling

- Only the chunks inside the Render Distance are drawn
 - Since they are the only still existing
- Applied Frustum Culling
- Terrain texture has different textures depending on the height

Texture e Shader

- Readapted a PBR shaders to use textures and lighting
- An MRA image is created using Roughness and Ambient Occlusion
 - From the loaded textures
 - Metalness is set to 0
- Then is used by the shaders together with a Normal map

- PBR shader used for lightning interaction with textures
- A shader is used for the trees another for the terrain
 - Separation is needed since terrain use two textures
- Light class has been made for create directional light
 - Rapresenting the sun
 - Can manage more light by the shaders

Player

- Class that manage the camera and player controls
- Use a semi-realistic physics with different parameters a.
 - Gravity
 - Air drag
 - Ground drag
 - Bobbing
- Has the attribute of the Frustum Planes
 - Needed for frustum culling